



Virtualization vs Containerization

1. المشكلة الأساسية قبل ظهور Virtualization (الافتراضية)

- **قصة التخيل:** تخيل أنك مسؤول سيرفر في شركة والديفلوبر (المطور) أنجز كوداً يريد تشغيله، واكتشفت أن الكود يحتاج للعمل حصراً على نظام تشغيل **Ubuntu Linux**.
- **التنفيذ:** قمت ببساطة بتثبيت كل ما يتطلبه الكود على السيرفر الذي يعمل بنظام **Ubuntu**، وعمل البرنامج بنجاح تام.
- **المشكلة الجديدة:** بعد فترة، جلب المطور ميزة جديدة (Feature) يتطلب تشغيلها بيئة توزيع أخرى تماماً وهي **Red Hat**، بينما السيرفر الأساسي يعمل بنظام **Ubuntu** فقط.
- **الحل التقليدي قديماً:** لكي تشغل توزيع **Red Hat** بجانب **Ubuntu**، اضطرروا قديماً لشراء سيرفر مادي جديد بالكامل لتثبيت **Red Hat** عليه وتشغيل التطبيق الجديد، وهنا ظهرت الحاجة الملحة لحل هذه الأزمة الاقتصادية والتقنية.

2. ظهور الفيجواليزيشن (Virtualization - الافتراضية)

- **المفهوم:** بدلاً من شراء سيرفر جديد، ظهرت تقنية ال **Virtualization** لتمكنك من تشغيل نظام تشغيل فوق نظام تشغيل آخر.
- **طريقة العمل:**
 - لديك السيرفر الأساسي (الهوست) يعمل بنظام **Ubuntu**.
 - تقوم بتثبيت برنامج يُسمى **Hypervisor** (مثل: **VMware**، **Hyper-V**، أو **KVM**).
 - يسمح لك هذا البرنامج بإنشاء أجهزة وهمية تسمى **VMs** (Virtual Machines)، بحيث يمكنك إنشاء **VM** بنظام **Ubuntu** و **VM** أخرى بنظام **Red Hat**، وتشغيل التطبيقات المختلفة عليها في نفس الوقت على نفس السيرفر.

3. المشاكل والعيوب التي واجهت الـ Virtualization

على الرغم من أن الـ VMs حلت مشكلة تشغيل أنظمة مختلفة، إلا أنها أفرزت 3 مشاكل رئيسية للإدارة (Operations):

- أ. وقت الإقلاع الطويل (Boot Time): عند عمل إعادة تشغيل (Restart) للسيرفر، يحتاج النظام لتحميل نظام التشغيل الأساسي، ثم تشغيل الهايبرفايزور، ثم قيام كل VM بتحميل نظام التشغيل الخاص بها من الصفر، وهو أمر بطيء ويأخذ وقتاً طويلاً تماماً مثل إقلاع ويندوز.
- ب. إهدار الموارد وإدارتها (Resource Management):
 - عند إنشاء VM، تقوم بحجز وتثبيت جزء محدد من موارد السيرفر الأصلية (مثل تخصيص 2 جيجا رام و Core 2 من المعالج).
 - إذا كان التطبيق الأول يحتاج 1 جيجا فقط، ستبقى الـ 1 جيجا الباقية محجوزة ومجمدة ولا يمكن لتطبيق آخر الاستفادة منها، حتى لو كان التطبيق الثالث بحاجة ماسة لمزيد من الموارد وكاد أن يحدث له كراش (Crash)، فلن يستطيع تخطي الحد المخصص له أو أخذ الموارد الفائضة من غيره، مما يؤدي لإهدار الموارد.

ج. ضخمة الحجم وصعوبة النقل (Image Size & Migration):

- عند نقل الـ VMs إلى سيرفر جديد بمواصفات أعلى، يجب أن يكون الـ Hypervisor متوافقاً.
- لتحويل الـ VM ونقلها، يتم أخذ نسخة مطابقة تشبه ملف الـ ISO تُسمى Image للـ VM.
- حجم هذه الـ Image يكون ضخماً جداً (قد يصل إلى 10 جيجا بايت كحد أدنى للـ VM الواحدة) لأنها تحوي نظام التشغيل الكامل، واللايبراري، والبرامج، والتطبيق. إذا كان لديك VM 100، فأنت مطالب بنقل حوالي 1 تيرابايت كاملة وإعادة تثبيتها، وهو أمر شاق وغير عملي.

4. ظهور الحل الجديد: الكونترايزيشن (Containerization)

- **المفهوم:** لحل مشاكل الـ Virtualization الثقيلة، ظهر حل جديد كلياً وهو Containerization (الحاويات).
- **طريقة العمل:**
 - على السيرفر الأساسي (الهوست) الذي يعمل بنظام Ubuntu، لا نقوم بتثبيت Hypervisor ولا نظام تشغيل كامل، بل نقوم بتنزيل محرك حاويات يُسمى Container Runtime / Engine (مثل Docker).
 - يتيح لك دوكر إنشاء ما يُسمى Container (الحاوية)، وهي تشبه الـ VM تماماً في أداء الغرض ولكن مع استبعاد جزء أساسي منها.

5. كيف تفهم فلسفة عمل الكونتینر ومشاركة النواة (Kernel)؟

- سؤال بديهي: كيف يمكنني تشغيل تطبيق يحتاج توزيعه ريد هات أو بيئة مختلفة بينما السيرفر الأساسي يعمل بنظام آخر؟ أليس الكونتینر لا يحتوي على نظام تشغيل؟
- الحقيقة التقنية:
- كل توزيعات لينكس (سواء Red Hat، Ubuntu، Debian، أو غيرها) تعتمد في جوهرها على شيء واحد مشترك يُسمى Linux Kernel (نواة لينكس).

1. ما هي نواة نظام التشغيل (OS Kernel) أصلاً؟

- النواة هي "القلب النابض والعقل المدبر" لأي نظام تشغيل. هي الطبقة البرمجية العميقة المسؤولة عن التواصل المباشر مع الهاردوير (مثل المعالج CPU، الذاكرة RAM، كرت الشاشة، والهارديسك).
- أي برنامج أو تطبيق تريده أن يتعامل مع قطع الجهاز، يجب أن يمر عبر هذه النواة لكي تفهم الأوامر وترجمتها للعتاد.
- ما الذي يختلف إذن بين توزيعه وأخرى؟ الاختلاف يكمن فقط في:
- البرامج والباينريز (Binaries & Programs): مثل نسخة بايثون أو الأدوات المساعدة المرفقة.
- نظام الملفات (File System): طريقة تنظيم وترتيب المجلدات والدايركتوري.

○ دور Docker في تجاوز المشكلة:

- الكونتینر يتخلص تماماً من نظام التشغيل الكامل ومن النواة (Kernel)، ويقول للجميع: "سنستخدم جميعاً نواة نظام التشغيل الأساسي (Host Kernel)".
- يقوم دوكر بإنشاء الكونتینر بحيث يحتوي فقط على:
- التطبيق ذاته (مثل كود بايثون).
- البرامج والمكتبات اللازمة فقط لتشغيل هذا التطبيق (مثل نسخة بايثون وال Binaries الخاصة بها).
- نظام الملفات (File System) الخاص بالتطبيق.
- بعد ذلك، يربط دوكر هذه المكونات بنواة النظام الأساسي (Host Kernel) مباشرة لتتشاركها الحاويات فيما بينها (Shared Kernel).
- المتحكم الأساسي: Docker أو (Container Runtime) هو البرنامج الذي يتدخل ويدير عملية تشغيل الحاويات ويسمح لها بمشاركة نواة النظام الأساسي بسلاسة وأمان تام دون الحاجة لتكرار أنظمة تشغيل كاملة.

3. ما المقصود بـ "أن دوكر يستغل نواة نظام التشغيل الأساسي مباشرة"؟

- في أنظمة Linux الأصلية:
حاوية دوكر (Container) هي في جوهرها مجرد عملية عادية (Process) تعمل على النظام، لكن تم عزلها بذلك شديد باستخدام ميزات أمان وعزل داخل نواة لينكس نفسها. لذلك، الحاوية لا تحتاج لنواة خاصة بها، بل تستخدم نواة اللينكس الأساسية لجهازك مباشرة لتنفيذ العمليات، وتكتفي بتضمين ملفات التطبيق ومكتباته فقط.
- الوضع على نظام Windows (وعبر تقنية WSL2):
 - بما أن نظام Windows يعتمد على نواة مختلفة تماماً عن لينكس (نواة ويندوز)، فكيف تعمل حاويات لينكس عليه؟
 - هنا يأتي دور WSL2؛ فهي توفر نواة لينكس حقيقية وخفيفة تعمل في خلفية الويندوز.
 - عندما تشغل حاوية دوكر مبنية على لينكس على جهازك الويندوز، فإن دوكر يوجه طلبات الحاوية مباشرة إلى نواة اللينكس الموجودة في خلفية الـ WSL2.

○ ما الذي يفعله WSL2؟

- قامت شركة Microsoft بالتعاون مع مطوري لينكس بدمج نواة لينكس حقيقية ومصغرة (Real Linux Kernel) داخل نظام Windows نفسه.
- عندما تفعل WSL2، يصبح ويندوز قادراً على تشغيل برمجيات وأوامر اللينكس مباشرة وبأعلى أداء دون الحاجة لتثبيت نظام تشغيل كامل بجانب الويندوز أو الاستعانة ببرامج بطيئة مثل VirtualBox.

💡 الخلاصة ببساطة:

- الـ VM: كل جهاز افتراضي يأتي بنواته المستقلة الثقيلة الخاصة به (تعدد وتكرار للنواة حسب عدد الأجهزة).
- Docker + WSL2: هناك نواة واحدة مركزية خفيفة في الخلفية، وكل الحاويات تتشارك وتستعير هذه النواة الوحيدة للتعامل مع النظام، وهذا هو السر في أن دوكر يستهلك موارد أقل بكثير من الـ VMs!

3. كيف يحل Docker مشاكلنا في الذكاء الاصطناعي و MLOps؟

1. القضاء على أعذار (It works on my machine):

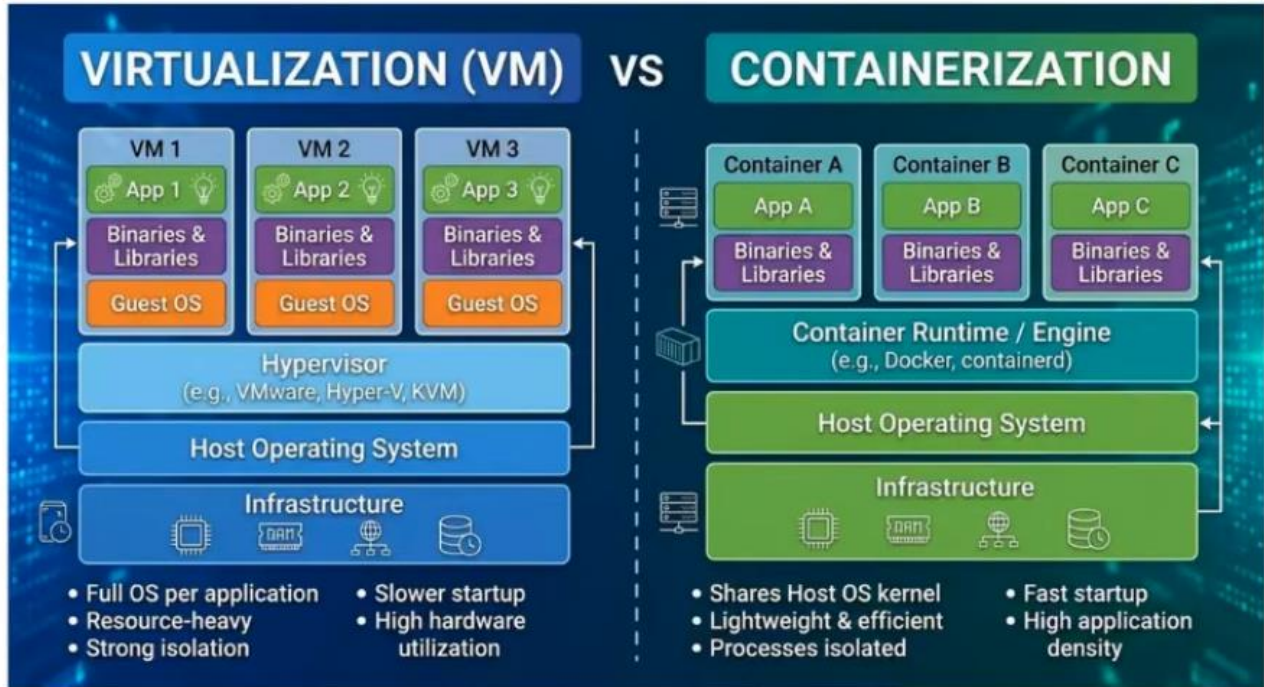
عندما تضع نموذج ال Computer Vision الخاص بك ونظام ال FastAPI داخل حاوية Docker، فأنت تضمن أن البيئة البرمجية التي جربت عليها الموديل هي نفسها تماماً التي ستعمل على سيرفر الإنتاج في الشركة.

2. خفة الوزن (Lightweight):

على عكس الأجهزة الوهمية التقليدية (Virtual Machines) التي كانت تستهلك مساحة ضخمة من الرامات والهارديسك لأنها تحمل نظام تشغيل كامل، فإن Docker يشارك نظام التشغيل الأساسي لجهازك (عبر WSL2) وبالتالي الحاويات تكون سريعة جداً وتفتح في أجزاء من الثانية.

CONTAINERS AND DOCKER

Two Ways to Isolate an App



إليك أسئلة وأجوبة تفصيلية مبنية على النص وموضحة بالمصطلحات التقنية:

○ س: كيف يؤثر استخدام Docker على سرعة الإقلاع (Boot Time) مقارنة بال VMs؟

○ ج: الإقلاع في دوكر يكون أسرع بكثير؛ لأنه لا يوجد سوى نظام تشغيل واحد (Host Operating System) يقوم بالعمل، يليه تشغيل محرك دوكر (Docker)، وكل Container يحتوي فقط على البرامج البسيطة التي يحتاجها للتشغيل (مثل بايثون أو Node.js فقط) وليس نظام تشغيل كاملاً.

○ س: كيف يتعامل الـ Container مع الـ Resources (الموارد) وكيف يختلف عن الـ VM في إدارة الذاكرة والمعالج؟

○ ج: بما أن الـ Container يشارك نواة النظام (Host Kernel)، فهو يرى نفسه ظاهرياً أنه يمتلك كل موارد الجهاز ولا يوجد معه أحد. ولكن لتجنب استنزاف السيرفر، يتم وضع Limits (حدود) استهلاك لكل حاوية. وعلى عكس الـ VM الذي يحجز موارد ثابتة لا يمكن لغيره استغلالها حتى لو كانت مهدرة، فإن حاويات دوكر المرنة يمكنها الاستفادة من الموارد المتاحة أو الفائضة من حاويات أخرى إذا دعت الحاجة وتحت الحدود المسموحة.

○ س: كيف يتعامل الـ Container مع الـ Resources (الموارد) وكيف يختلف عن الـ VM في إدارة الذاكرة والمعالج؟

○ ج: بما أن الـ Container يشارك نواة النظام (Host Kernel)، فهو يرى نفسه ظاهرياً أنه يمتلك كل موارد الجهاز ولا يوجد معه أحد. ولكن لتجنب استنزاف السيرفر، يتم وضع Limits (حدود) استهلاك لكل حاوية. وعلى عكس الـ VM الذي يحجز موارد ثابتة لا يمكن لغيره استغلالها حتى لو كانت مهدرة، فإن حاويات دوكر المرنة يمكنها الاستفادة من الموارد المتاحة أو الفائضة من حاويات أخرى إذا دعت الحاجة وتحت الحدود المسموحة.

○ س: ما هي الفائدة من وضع Limits (حدود) لموارد الـ Container رغم أن ذلك قد يقيّد استخدامه؟

○ ج: الفائدة الرئيسية أمنية (Security) وفيزيائية لمنع انهيار السيرفر بالكامل؛ فلو تعرض أحد الـ Containers للاختراق وتم استغلاله لسحب كل الموارد دون Limit ، فسيتوقف السيرفر بأكمله (Down). كما أن تحديد اليميت يساعد في عملية الـ Monitoring (المراقبة) وتوزيع الموارد بكفاءة بين الـ Containers المختلفة حتى لا يقوم كونتينر واحد باستهلاك كل شيء وإسقاط باقي التطبيقات.

○ س: لماذا يُعد حجم الـ Image الخاصة بالكونتينر صغيرة جداً (تقاس بالميجابايت) مقارنة بال VMs؟

○ ج: لأن الـ Container Image لا تحتوي على نظام تشغيل كامل (Operating System) كما هو الحال في الـ VMs (التي قد يصل حجمها لـ 10 أو 20 جيجابايت)، بل تعتمد كلياً على الـ Kernel الخاص بالـ Host ، ولا تنزل سوى البرامج والباينريز الدقيقة التي يحتاجها التطبيق فقط، مما يسهل تخزينها ومشاركتها ونقلها.

Docker (Docker Architecture)

1. البنية المعمارية لـ Docker (Docker Architecture)

عندما تقوم بتثبيت دوكر على جهازك، فهو لا يعمل كبرنامج واحد، بل يتكون من جزئين أساسيين يتواصلان معاً (طريقة عمل العميل والخادم Client-Server Architecture):

- أ. واجهة أوامر دوكر أو العميل (Docker CLI / Docker Client):
 - هو البرنامج أو النافذة التي تكتب فيها أوامر دوكر (مثل: `docker run`). الكلاينت هنا لا ينفذ الأمر بنفسه، بل يترجمه ويبحث به إلى السيرفر.
- ب. محرك دوكر أو الخادم (Docker Engine / Daemon):
 - هو برنامج خفي (Background Service) يُسمى تقنياً (`dockerd`) يعمل دائماً في الخلفية و ينتظر أي أمر يأتيه من ال Client لكي ينفذه فوراً (مثل إنشاء حاوية، إيقافها، أو جلب صور).

2. كيف يتواصل العميل (Docker Client) مع الخادم (Docker Engine)؟

هناك طريقتان لهذا التواصل بناءً على مكان وجودك:

- الطريقة الأولى (التواصل المحلي - Local / Unix Socket & IPC):
 - إذا كان ال Client وال Server على نفس الجهاز، فهما لا يحتاجان لإنترنت أو بروتوكول شبكة معقد. يتواصل البرنامجان عبر قناة اتصال داخلية في النظام تُسمى Socket (وتحديداً عبر تواصل العمليات الداخلية IPC - Inter-Process Communication) لنقل الأوامر بسرعة فائقة بين العمليات على نفس ال Machine.
- الطريقة الثانية (التواصل عن بُعد - Remote / TCP & HTTP):
 - هل يمكنك التحكم في دوكر الموجود على سيرفر الشركة وأنت في البيت؟ نعم بالتأكيد.
 - يتم ذلك باستخدام بروتوكول HTTP / HTTPS عبر شبكة الإنترنت. ترسل الأمر من جهازك مع تحديد عنوان السيرفر (IP Address) واستخدام منفذ الاتصال المخصص لدوكر وهو البورت 2375 (عن طريق كتابة أمر محدد مثل `docker -H tcp://<IP>:2375`) ليتلقى ال Server Request و ينفذه عن بُعد.

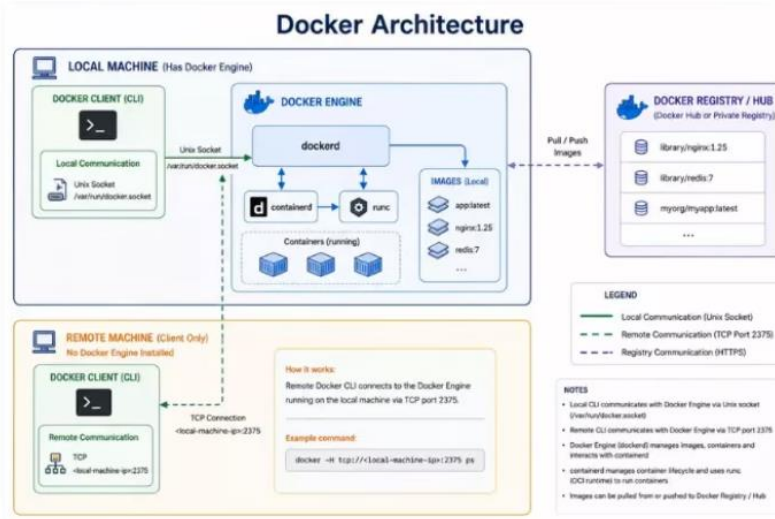
إليك الملخص الشامل والدقيق لمكونات دوكر وعلاقتها ببعضها ومن يقع بداخله:

- **ال Docker CLI (العميل):** هو واجهة أوامر الطرفية الخارجة التي تستخدمها أنت لكتابة الأوامر (مثل `docker run`). **هو برنامج منفصل وخارج ال Docker Engine تماماً**، وظيفته فقط التقاط أمرك وإرساله عبر ال Socket أو الشبكة.
- **ال Docker Engine (المحرك والسيرفر الشامل):** هو الهيكل البرمجي المتكامل الذي يستقبل طلبات ال CLI ويدير كل شيء. ويضم بداخله المكونات الأساسية التالية:
- **ال dockerd (Docker Daemon):** هو الخادم الأساسي (Server) والقلب النابض داخل ال Docker Engine، ويظل يعمل في الخلفية ليستقبل طلبات ال CLI ويدير الكائنات والصور والشبكات.
- **ال containerd (محرك الحاويات):** هو المكون الموجود داخل ال Docker Engine (والذي يعتمد عليه ال dockerd) ليتولى إدارة دورة حياة الحاويات والشبكات وملفات التخزين.
- **ال runc:** هو الأداة التنفيذية منخفضة المستوى الموجودة في طبقات التشغيل السفلى، والتي يستعين بها ال containerd لتتحدث مباشرة مع نواة نظام التشغيل (Kernel) لإتمام العزل وإنشاء الحاوية.

3. رحلة الأمر داخل دوكر (خطوات تنفيذ الأوامر)

- **كتابة الأمر في ال CLI (العميل):** تبدأ العملية عندما تقوم بكتابة أمر دوكر في الطرفية (مثل: `docker run nginx`) عبر ال Docker Client أو ال CLI (الذي يعد منفصلاً عن ال Docker Engine).
- **إرسال الطلب إلى dockerd (السيرفر):** يأخذ ال Client هذا الأمر ويحوّله إلى طلب بروتوكولي عبر ال Socket محلياً أو عبر شبكة الإنترنت) ليرسله إلى الخادم الأساسي والقلب النابض داخل ال Docker Engine والمعروف بـ `dockerd`.
- **التحقق من وجود ال Image محلياً:** يتلقى `dockerd` الطلب، ويبحث في المستودع المحلي للجهاز ليتحقق مما إذا كانت الصورة المطلوبة (Image) موجودة أم لا.
- **سحب الصورة من Docker Hub (إن لم توجد محلياً):** إذا لم يجد السيرفر ال Image محلياً، فإنه يتواصل مع Docker Hub عبر الإنترنت ليسحب الصور المطلوبة ويخزنها محلياً.
- **تمرير الطلب إلى containerd (محرك الحاويات):** بعد توفر ال Image والجاهزية، يقوم `dockerd` بتمرير الأمر إلى محرك الحاويات الموجود بداخله والمسمى `containerd` لتولي إدارة دورة حياة الحاويات والشبكات.
- **التواصل مع النواة عبر الأداة runc:** يستعين `containerd` بأداة التنفيذ السفلى `runc`، وهي المسؤولة عن مخاطبة نواة نظام التشغيل (Kernel) مباشرة لإتمام عزل العملية وبناء الحاوية لتصبح حية وتعمل بنجاح.

Docker Architecture



How a command flows

- 01 The CLI sends your command to the Docker daemon
- 02 The daemon builds images and manages containers
- 03 containerd and runc start the container process
- 04 A registry stores images, ready to pull or push

1. ربط ما سبق: دور ال `containerd` في إنشاء ال Container

تحدثنا سابقاً أنه بعد أن يقوم ال `dockerd` بالتحقق من ال Image (أو سحبها من Docker Hub)، فإنه يمرر الأمر إلى ال `containerd` (محرك الحاويات).
الآن سنرى بدقة ماذا يفعل ال `containerd` خلف الكواليس لتلك الحاوية وكيف يحولها إلى حقيقة على مستوى نظام التشغيل.

2. كيف ينشئ ال `containerd` ال Container؟ (بنية العمل الداخلية)

ال Container في جوهره ليس جهازاً وهمياً، بل هو عملية (`Process`) عادية جداً تعمل على نواة نظام التشغيل، ولكنها مُحاطة بتقنيات عزل قوية تجعلها تبدو وكأنها نظام مستقل تماماً. يقوم ال `containerd` بالخطوات التالية لإنشائها:

- أ. إنشاء الفايل سيستم الخاص بال Container (الملفات والدايركتوري):
 - يقوم ال `containerd` بتجهيز مساحة أو مجلد مخصص في نظام الملفات الخاص بال Host، ويضع بداخله فقط الملفات والدايركتوري والبرامج التي يحتاجها التطبيق (مثل كود بايثون ومكتباته)، دون الحاجة لنظام تشغيل كامل.
- ب. توليد ال `Process` (عملية التشغيل):
 - يقوم ال `containerd` بإنشاء عملية (`Process`) جديدة تتفرع منه (تعتبر كأنها "ابن" أو Child `Process` له)، ويتم داخل هذه العملية تشغيل البرنامج المطلوب (مثل تشغيل أمر `nginx` أو ملف `python.py`).

3. تقنيات العزل الأساسية (Namespaces و Cgroups)

لكي تكون هذه الـ Process معزولة وآمنة تماماً، يعتمد النظام (عبر الـ `containerd` والأدوات السفلى مثل `runc`) على تقنيتين أساسيتين في نواة اللينكس (وهما موجودتان في اللينكس أساساً قبل ظهور Docker) وتم استغلالهما لتنفيذ فكرة الـ Container:

أ. الـ Namespaces (مساحات الأسماء - للعزل الكامل):

- هي التقنية المسؤولة عن عزل الـ Container تماماً عن بقية الجهاز.
- تجعل الـ Container لا يرى أي برامج أخرى تعمل على الـ Host، ولا يرى ملفات الحاويات الأخرى.
- يرى البرنامج داخل الـ Container نفسه على أنه المسيطر الوحيد وأنه الجهاز بأكمله، ولا يستطيع التواصل إلا مع العمليات التي تخصه فقط ضمن نفس الـ Namespace.
- داخل الـ Container: يجعلك دوكر تشعر أنك تستخدم الـ `Root` لكي تتمكن من تثبيت البرامج وتشغيل أدواتك داخل الحاوية براحتك دون مشاكل.
- على مستوى جهازك الأساسي (Host): أنت في الحقيقة تعمل بصلاحيات مستخدم عادي جداً ومعزول.
- الهدف الأمني العظيم: هذه الحيلة الذكية تحمي جهازك الأساسي تماماً؛ فلو افترضنا أن شخصاً ما استطاع اختراق التطبيق داخل الحاوية وعمل باعتباره `Root`، فسوف يتفاجأ بأنه "ملك على لا شيء" خارج الحاوية، ولن يقدر على إيذاء ملفات جهازك الأساسي أو اختراقه، لأنه في الحقيقة مسجون داخل حدود الصلاحيات المعزولة!

ب. الـ Cgroups (Control Groups - للتحكم في الموارد):

- هي التقنية المسؤولة عن تحديد ومراقبة الموارد (مثل تحديد حد أقصى للـ CPU، الـ Memory، أو عمليات الـ I/O).
- كما شرحنا سابقاً، إذا لم تقم بوضع Limits عبر الـ Cgroups، فإن الـ Process الخاصة بالـ Container ستستهلك كل الموارد المتاحة في الجهاز إذا احتاجت لذلك. أما بوضع الحدود، فإن الـ Cgroups تضمن عدم تجاوز الـ Container للحد المسموح به ومنع انهيار السيرفر.

Namespaces and cgroups

namespaces decide what a container can see, cgroups
decide how much it can use

○ ما هو الفرق الجوهرى بين إدارة الـ Resources فى الـ VM (الـ Virtual Machines) مقارنة بـ الـ Containers ؟

○ فى الـ VM ، يتم حجز الموارد وتثبيتها مسبقاً (Static Allocation) بشكل كامل حتى لو لم يستهلكها التطبيق، فلا يمكن لتطبيق آخر الاستفادة من الموارد الفائضة. أما فى الـ Containers ، فهناك مرونة عالية (Flexibility) فى مشاركة وإدارة الموارد (Sharing Resources) بحيث يمكن استغلال المساحات الفائضة طالما لم يتم تجاوز الحد الأقصى المسموح به.

○ ما هو المقصود بمفهومي الـ Request والـ Limit عند تحديد موارد الـ Container ؟

○ الـ Request هو الحد الأدنى أو المبدئى للموارد الذى يطلبه التطبيق لكي يعمل بشكل طبيعى، بينما الـ Limit هو الحد الأقصى والسماحية الإضافية التى يمكن أن يصل إليها الـ Container فوق الـ Request بناءً على اتفاق مسبق مع الـ Developer وحسب طبيعة الـ Business وحجم الـ Load المتوقع.

5. دور الـ containerd كأب للـ Processes (دورة حياة الـ Container)

○ بما أن الـ containerd هو من قام بتوليد وإنشاء عملية الـ Container كـ "ابن" (Child Process) له، فهو المسؤول المباشر عن إدارتها ومتابعتها (Management).

○ ماذا يحدث لو توقف البرنامج الأساسى داخل الـ Container؟

○ إذا كان الـ Container مخصصاً لتشغيل برنامج معين (مثل سكربت بايثون) وانتهى هذا البرنامج أو حدث له كراش (Crash)، فإن الـ Process تنتهى.

○ ونتيجة لذلك، ينتهى عمر الـ Container فوراً ويغلق، لأنه لا يوجد نظام تشغيل وهمى خلفه يعمل بلا طائل؛ الـ Container مرتبط ارتباطاً وثيقاً بوجود البرنامج يعمل بداخله.

6. أفضل ممارسات دوكر (Best Practices): لماذا لا نضع عدة برامج فى Container واحد؟

○ المفهوم الخاطئ: قد يظن البعض أنه يمكن وضع عدة تطبيقات وبرامج ضخمة داخل نفس الـ Container تماماً مثل الـ VM.

○ التصحيح والأفضلية: هذا غير مستحب نهائياً. فلسفة الـ Containers تقوم على مبدأ Microservices (الخدمات المصغرة)، حيث يُخصص كل Container لخدمة أو تطبيق واحد فقط.

○ السبب: إذا وضعت أكثر من برنامج فى نفس الـ Container وتوقف أحدهم أو حدثت مشكلة، فلن تتمكن من مراقبتها أو إدارتها بشكل صحيح، وستفقد ميزة السهولة والعزل المرن التى يقدمها دوكر.

1. ما هي الـ Image وكيف يتم بناؤها؟

- **التعريف:** الـ Image هي قالب أو كتالوج تعليمات ثابت (`Read-Only Layers, built once`) يتم بناؤه مرة واحدة فقط، وتحتوي على كافة الإرشادات لتشغيل التطبيق (مثل اختيار إصدار بايثون، نسخ الملفات، تثبيت المكتبات، وتحديد أمر التشغيل).
 - **طريقة البناء عبر الطبقات (Layers):** تُبنى الـ Image بشكل تراكمي في صورة طبقات فوق بعضها بناءً على ملف الـ `Dockerfile` (كما يظهر في الصورة):
 - **Layer 1 (`FROM python:3.12-slim`):** الطبقة الأساسية (`Base Image`) التي تنزل نظام التشغيل الخفيف مع بيئة بايثون.
 - **Layer 2 (`COPY requirements.txt .`):** طبقة نسخ ملف المتطلبات إلى داخل النظام.
 - **Layer 3 (`RUN pip install -r requirements.txt`):** طبقة تثبيت المكتبات المطلوبة.
 - **Layer 4 (`COPY . + CMD ["python", "app.py"]`):** طبقة نسخ باقي كود التطبيق وتحديد أمر التشغيل الأساسي.
 - **خصائصها:** كل هذه الطبقات داخل الـ Image تكون **قراءة فقط (Read-Only)**، ولا يمكن تعديل ملفاتهما بشكل مباشر بعد بنائها.
-

- **الـ Dockerfile (وصفة العمل):** هو ببساطة ملف نصي يكتبه المطور ويحتوي على مجموعة من التعليمات والخطوات المكتوبة بترتيب محدد (مثل اختيار لغة البرمجة، نسخ الملفات، وتثبيت المكتبات).
- **عملية التحويل (`docker build`):** عندما تقوم بتنفيذ أمر البناء، يأخذ دوكر ملف الـ Dockerfile ويقرأ تعليماته سطرًا بسطر لتنفيذها وتوليد المنتج النهائي.
- **الـ Image (المنتج النهائي والطبقات):** النتيجة المباشرة لتنفيذ تعليمات الـ Dockerfile هي تكوين الـ Image، حيث تتحول كل تعليمة في الملف إلى **طبقة مستقلة وثابتة (Read-Only Layer)** تُبنى فوق بعضها البعض بشكل تراكمي.
- **باختصار:** الـ Dockerfile هو "الكود أو النص المكتوب"، بينما الـ Image هي "النتيجة العينية المبنية" والمكونة من عدة طبقات جاهزة لإطلاق الحاويات منها في أي وقت.

2. ما هو ال Container؟ وكيف يختلف عن ال Image؟

- **التحول إلى Container:** عندما تقوم بتنفيذ أمر التشغيل (`docker run`)، فإن دوكر يأخذ هذه ال `Image` الثابتة ويقوم بتشغيلها.
- **ال Writable Layer (طبقة الكتابة المؤقتة):** بما أن ال `Image` أصلها `Read-Only`، فماذا يحدث لو قام التطبيق أثناء عمله بإنشاء ملفات جديدة، أو تعديل بيانات، أو تسجيل سجلات (`Logs`)؟
 - يقوم دوكر بوضع طبقة إضافية قابلة للكتابة (`Writable Layer`) فوق طبقات ال `Image` الثابتة.
 - هذه الطبقة الإضافية تكون رقيقة وخفيفة (`A thin layer`)، ويتم فيها تخزين أي تعديلات أو ملفات جديدة يكتبها التطبيق أثناء تشغيله.
- **مصير هذه الطبقة:** هذه الطبقة المؤقتة مرتبطة بدورة حياة ال `Container`؛ فإذا تم حذف ال `Container`، يتم حذف هذه الطبقة المؤقتة وما عليها من تعديلات غير محفوظة. (ولحفظ البيانات الهامة بشكل دائم، يتم استخدام ما يُسمى بـ `Volumes` كما يشير التوجيه في أسفل الصورة).

IMAGE

read-only layers, built once

LAYER 4
COPY . . + CMD ["python", "app.py"]
LAYER 3
RUN pip install -r requirements.txt
LAYER 2
COPY requirements.txt .
LAYER 1
FROM python:3.12-slim

`docker build` → layers that never change

CONTAINER

the same layers plus one writable layer

WRITABLE LAYER
logs, temporary files, anything the app writes
LAYER 4
COPY . . + CMD ["python", "app.py"]
LAYER 3
RUN pip install -r requirements.txt
LAYER 2
COPY requirements.txt .
LAYER 1
FROM python:3.12-slim

`docker run` → a thin layer, removed with the container

Many containers can share one image. Data you want to keep belongs in a volume, not in the writable layer.



Image and Container Commands



COMMAND

WHAT IT DOES

`docker pull nginx:1.27`

download an image from a registry

`docker images`

list the images stored on this machine

`docker build -t myapp:1.0 .`

build an image from the Dockerfile in this folder

`docker run -d -p 8080:80 myapp:1.0`

start a container in the background and publish a port

`docker ps -a`

list all containers, running and stopped

`docker logs -f <id>`

watch the output of a container as it runs

`docker exec -it <id> sh`

open a shell inside a running container

`docker rm -f <id> · docker rmi <img>`

delete a container · delete an image

`docker ps` : لعرض قائمة ال Containers النشطة والتي تعمل حالياً فقط.

`<docker stop <id>` : لإيقاف تشغيل ال Container بشكل آمن.

`docker exec -it <id> sh` (أو `bash`): لفتح واجهة أوامر تفاعلية (`Shell`) داخل ال Container النشط بهدف الفحص أو ال `Debugging` .



Docker Hub and Image Tags

01

Registry

where images are stored; Docker Hub is the public one, and private ones work the same way

|

02

Repository

a named group of images, for example myorg/myapp

03

Tag

the version label on an image, for example myapp:1.4.2

04

latest is not the newest

it is only the default tag and it keeps moving, so always name a real version in production

05

Digest

myapp@sha256:... points to one exact image, so every deployment is identical

- **ال Registry:** هو المخزن العام أو المنصة التي تُحفظ وتُخزن فيها ال Images، ويُعد Docker Hub هو ال Registry العام والأشهر، كما توجد سجلات خاصة (Private Registries) للشركات والأفراد لحفظ الصور بسرية وأمان.
- **ال Repository:** هو عبارة عن مجموعة مسماة من ال Images التي تخص تطبيقاً واحداً ولكن بفيرجينات أو إصدارات مختلفة (مثل python أو myorg/myapp)، ويمكن أن يكون عاماً (Public) أو خاصاً (Private).
- **ال Tag (الإصدار):** هو التسمية أو العلامة التي تُوضع على ال Image لتحديد إصدارها بدقة (مثل 1.4.2 أو إصدارات ال Alpine الخفيفة)، ويفصل بين اسم ال Repository ورقم الإصدار بشرطة راسخة أو نقطتين كولون (:).
- **مفهوم latest ليس بالضرورة الأحدث:** كلمة latest هي ببساطة ال Default Tag الافتراضي الذي يتم توجيهه تلقائياً عند عدم كتابة رقم الإصدار، ولا تعني بالضرورة أنها أحدث نسخة تم إطلاقها، ولذلك يُفضل دائماً تحديد رقم الإصدار الحقيقي بدقة في بيئات ال Production.
- **ال Digest:** هو المعرف الفريد المشفر (مثل بصمة ال sha256) الذي يشير إلى نسخة مطابقة تماماً ودقيقة من ال Image لضمان تطابق وثبات عملية ال Deployment في كل مرة.



CONTAINERS AND DOCKER

Working with a Registry



COMMAND

WHAT IT DOES

`docker login`

sign in to Docker Hub or a private registry

`docker tag myapp:1.0 me/myapp:1.0`

give the image a name that points to your repository

`docker push me/myapp:1.0`

upload the image and its layers

`docker pull me/myapp:1.0`

download the image on another machine

`docker image inspect me/myapp:1.0`

show the digest, the layers and the settings

`docker search postgres`

search public images from the command line



CONTAINERS AND DOCKER



Docker Network Types

01

bridge

the default private network on one host; publish ports with `-p 8080:80`

02

user-defined bridge

the same, and containers can also reach each other by name

03

host

no network isolation; the container uses the network of the host

04

none

no network at all, for tasks that work offline

05

overlay and macvlan

overlay joins containers across hosts; macvlan gives a container its own LAN address

○ أهمية شبكات دوكر (Docker Networks):

- تُستخدم لحل مشكلتين رئيسيتين: كيف يقوم مستخدم خارجي بالوصول إلى التطبيق الموجود داخل الـ Container ، وكيف تتواصل الـ Containers مع بعضها البعض (مثل اتصال تطبيق بايثون بقاعدة بيانات).

○ آلية شبكة الـ bridge الافتراضية:

- عند إنشاء أي Container ، يتم وضعه تلقائياً في شبكة الـ bridge الافتراضية إذا لم تحدد شبكة غيرها.
- في هذه الشبكة الافتراضية، تستطيع الـ Containers التواصل مع بعضها البعض، ولكن عن طريق الـ IP أو الـ ID الخاص بكل Container وليس عن طريق الأسماء.

○ الشبكات المخصصة (User-defined bridge):

- يمكنك إنشاء شبكة خاصة بك باستخدام الأمر `docker network create`.
- الميزة الكبرى هنا هي ميزة الـ DNS : فالـ Containers الموجودة داخل شبكة مخصصة تستطيع التواصل مع بعضها البعض باستخدام أسماء الـ Containers مباشرة بكل سهولة، بينما تظل معزولة تماماً عن أي Container في شبكة أخرى.

○ ربط المنافذ (Port Forwarding مع الهوست):

- لكي يتمكن مستخدم خارجي من الوصول إلى تطبيق يعمل داخل Container ، يتم ربط منفذ الجهاز الأساسي (Host) بمنفذ الـ Container عند تشغيله (باستخدام المعامل `-p 8080:80` مثلاً).
- هذا يعني أن أي طلب (Request) يأتيك على بورت معين في جهازك يتم توجيهه تلقائياً إلى داخل الـ Container .
- يتيح تشغيل عدة Containers على نفس المنفذ داخلياً دون تعارض عن طريق تغيير رقم البورت من جهة الهوست (مثل تشغيل Container على 8080 وآخر على 8081).

○ شبكة الـ Host:

- يلغي الـ Container العزل الشبكي تماماً ليستخدم شبكة الجهاز الأساسي (Host) مباشرة، بحيث تصبح منافذ الـ Container هي نفس منافذ الهوست دون الحاجة لعمل ربط (Port Forwarding).
- العيوب: يمنع تشغيل أكثر من Container على نفس المنفذ (مثل بورت 80) لتعارض الأجهزة وحجز المنفذ مسبقاً.

○ شبكة الـ None:

- يعزل الـ Container شبكياً بشكل كامل (مستقل بذاته)، فلا يكون متصلاً بأي شبكة أو بـ Containers أخرى، ولا يراها ولا يراه أحد.



Network Commands

COMMAND	WHAT IT DOES
<code>docker network ls</code>	list the networks on this machine
<code>docker network create appnet</code>	create your own bridge network with DNS
<code>docker run --network appnet ...</code>	connect a container when it starts
<code>docker network connect appnet web</code>	connect a container that is already running
<code>docker network inspect appnet</code>	show the subnet and the connected containers
<code>docker run -p 8080:80 nginx</code>	publish a container port on the host



Docker Storage Options

مشكلة تخزين البيانات في (Docker Storage Problem)

- السؤال: ماذا يحدث للبيانات والتعديلات داخل الـ Container إذا تم حذفه؟
- الإجابة: تفقد كل شيء؛ لأن الـ Container يمتلك الـ File System الخاص به، وبمجرد حذف الـ Container يتم مسح الـ File System بكل ما فيه من تعديلات نهائياً ولا يمكن استرجاعها بالطرق العادية.

خيارات التخزين في دوكر (Docker Storage Options)

لحل هذه المشكلة، توفر مساحة تخزين خارجية أو نربط مسارات محددة لكي تبقى البيانات ثابتة على الـ Host حتى لو تم حذف الـ Container، مما يتيح لأي Container جديد الوصول إلى نفس التعديلات القديمة مباشرة دون إعادة إنشائها من الصفر. وينقسم ذلك إلى ثلاثة أنواع رئيسية:

1. الـ Volume

- ما هو؟ هو مساحة تخزين يديرها Docker بالكامل (storage managed by Docker) ويضع لها اسمًا محددًا في مكان تخصيصه على الـ Host بعيداً عن التحكم المباشر للمستخدم.
- آلية العمل: تقوم بربط مسار معين داخل الـ Container (مثل app/) بهذا الـ Volume، بحيث أي تغيير يحدث داخله يُحفظ مباشرة ويبقى مستمراً بعد إزالة الـ Container.
- الاستخدام الشائع: الخيار الافتراضي والمفضل لقواعد البيانات (Databases) وأي بيانات تريد الاحتفاظ بها لفترة طويلة.

2. الـ Bind mount

- ما هو؟ هو ربط مجلد محدد من الـ Host (a folder from the host) مباشرة داخل الـ Container.
- آلية العمل: يتم ربط مسار أو دليل (Directory) في الـ Host بمسار آخر داخل الـ Container.
- الاستخدام الشائع: مفيد جداً أثناء مرحلة التطوير (Useful while developing)، لأن أي تعديل تبرمه على الكود في الـ Host يظهر داخل الـ Container في الحال دون الحاجة لإعادة بناء الـ Image.

3. الـ tmpfs

- ما هو؟ مساحة تخزين تعيش في الذاكرة العشوائية فقط (storage that lives in memory only) ولا تصل أبداً إلى الـ Disk.
- الاستخدام الشائع: مكان مناسب للملفات قصيرة الأجل (short-lived files) والبيانات الحساسة أو السرية (secrets) التي لا نريد الاحتفاظ بها بعد إيقاف التشغيل.



CONTAINERS AND DOCKER

Storage Commands

COMMAND	WHAT IT DOES
<code>docker volume create appdata</code>	create a named volume
<code>docker volume ls · inspect appdata</code>	list volumes · show where the data is kept
<code>docker run -v appdata:/var/lib/data</code>	mount that volume inside a container
<code>docker run -v \$(pwd):/app</code>	mount the current folder for development
<code>docker run --tmpfs /tmp:size=64m</code>	mount a folder that lives in memory
<code>docker volume prune</code>	delete the volumes no container uses



CONTAINERS AND DOCKER

Dockerfile: A Python App

```
Dockerfile x
01 # a small base image with a fixed version
02 FROM python:3.12-slim
03
04 WORKDIR /app
05
06 # install the dependencies first, so this layer is reused
07 COPY requirements.txt .
08 RUN pip install --no-cache-dir -r requirements.txt
09
10 # then copy the application code
11 COPY . .
12
13 ENV PORT=8000
14 EXPOSE 8000
15 USER 1000
16 CMD ["python", "app.py"]
```

- ما هو ال Dockerfile؟ هو ملف نصي نستخدمه لإنشاء Image خاصة بنا من الصفر أو بناءً على Base Image جاهزة (مثل صورة بايثون الرسمية)، لكي لا نبدأ من الصفر تماماً ونستفيد من نظام الملفات والبرامج الجاهزة فيها.
- تحديد مجلد العمل (WORKDIR): عند تحديد مسار مثل app/ داخل ال Container، يقوم دوكر بإنشائه والانتقال إليه تلقائياً ليكون مسار العمل الافتراضي لل Container.
- نسخ الملفات (COPY): يتم نسخ ملفات المشروع الضرورية مثل requirements.txt و ملف التطبيق الأساسي (مثل ملف بايثون) من ال Host إلى داخل مسار ال Image ليتم الاعتماد عليها.
- تثبيت الحزم (RUN): استخدام أمر التثبيت لتنزيل المكتبات والمتطلبات داخل بيئة ال Image المعزولة.
- ENV : يُستخدم لتحديد متغيرات بيئية داخل ال Container (مثل إعدادات الكونفجريشن أو المفاتيح السرية)، وتظل متاحة طوال فترة تشغيل ال Container.
- EXPOSE : يحدد المنفذ (Port) الذي سيستمع عليه التطبيق داخلياً، ويُعد بمثابة توثيق إرشادي (documentation) لتوضيح المنفذ المستخدم.
- USER : يحدد اسم المستخدم أو رقم ال UID الذي سيقوم بتنفيذ الأوامر وتشغيل التطبيق داخل ال Container لأسباب أمنية (مثل منع التشغيل بصلاحيات root).
- CMD : يحدد الأمر الافتراضي الذي يتم تنفيذه تلقائياً بمجرد إقلاع وتشغيل ال Container من ال Image.

Docker Compose: One File, Whole Stack

```
docker-compose.yml
01 services:
02   web:
03     build: .
04     ports: ["8000:8000"]
05     environment:
06       DB_HOST: db
07     depends_on: [db]
08
09   db:
10     image: postgres:16
11     environment:
12       POSTGRES_PASSWORD: secret
13     volumes:
14       - dbdata:/var/lib/postgresql/data
15
16 volumes:
17   dbdata:
```

مفهوم وإدارة متعدد الحاويات باستخدام Docker Compose

- المشكلة في الطريقة التقليدية: عند بناء تطبيق متكامل يتطلب أكثر من Container (مثل تطبيق Frontend، Backend، وقاعدة بيانات Database)، فإن تشغيل كل واحد بمفرده يتطلب كتابة أوامر `docker run` طويلة ومعقدة لكل Container على حدة، مع تذكر إعدادات الشبكات وال Volumes الخاصة بكل جزء.
- الحل باستخدام Docker Compose: يتم جمع كل هذه المكونات وترابطها داخل ملف تكوين واحد بصيغة YAML يسمى `docker-compose.yml`، لكي يتحكم في إدارة المنظومة كاملة من مكان واحد بدلاً من تشتت الأوامر.
- مفهوم الخدمات (Services): داخل ملف ال Docker Compose، تُعرف ال Containers تحت قسم يسمى `services`، حيث تمثل كل خدمة Container مستقلة (مثل خدمة `web` للواجهة وخدمة `database` لقاعدة البيانات)، مع إمكانية ربطها ببعضها ليتمكن ال Backend من التحدث مع ال Database بسلسلة عبر الشبكة.

```
docker compose up -d
```

لتشغيل كل المنظومة معاً بأمر واحد وبناء ما يلزم:

```
docker compose down
```

لإيقاف وحذف جميع ال Containers والشبكات المرتبطة بها دفعة واحدة:

```
docker compose down -v
```

لحذف ال Containers مع ال Volumes الخاصة بها تماماً لمنع هدر المساحة: